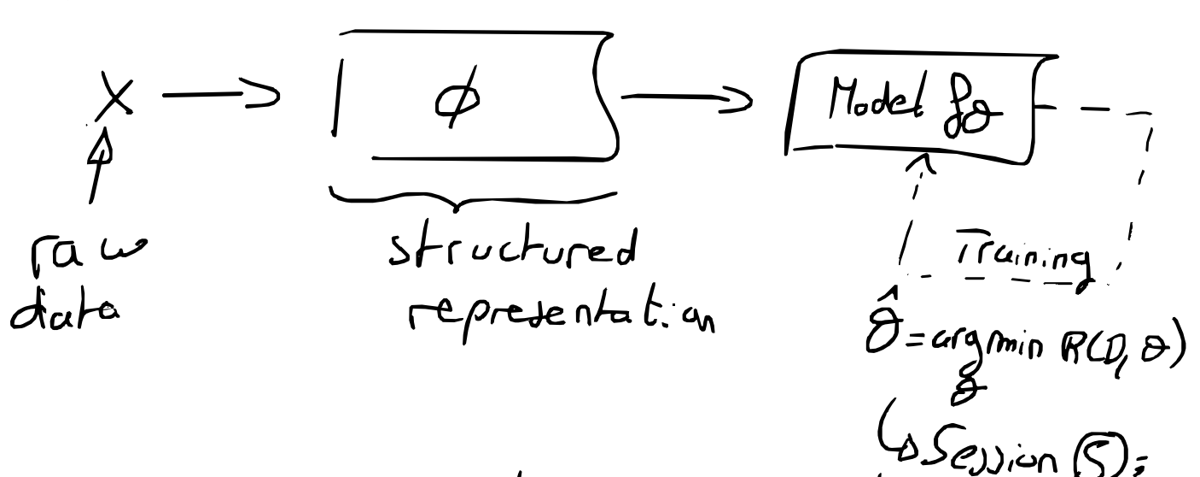
ML 4 SD

Session 4

Recap:

$x^{(i)} \in \mathbb{R}^d$ $\phi(x^{(i)}) \in \mathbb{R}^{d'}$ f_θ: parametrized function $\theta \in \mathbb{R}^p$


X
raw data
φ
structured representation
Model f_θ
Training
$\hat{\theta} = \arg\min_\theta R(D,\theta)$

↳ Session ⑤: optimization

① Specify a model

Ex:

f_θ: Linear Regression/Classification

Regression: $f_\theta(x^{(i)}) = \theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \dots$

Classification: $p_\theta = \frac{e^{\theta_k^T x}}{\sum_k e^{\theta_k^T x}}$ softmax model

① Cost function choice

$$R(D,\theta) = \frac{1}{n} \sum_{i=1}^{n} \underset{\substack{\uparrow \\ \text{loss}}}{\ell}(y^{(i)}, \hat{y}^{(i)})$$

Ex: $y \in \mathcal{Y}$ $\mathcal{Y}$: continuous → Regression, discrete → classification

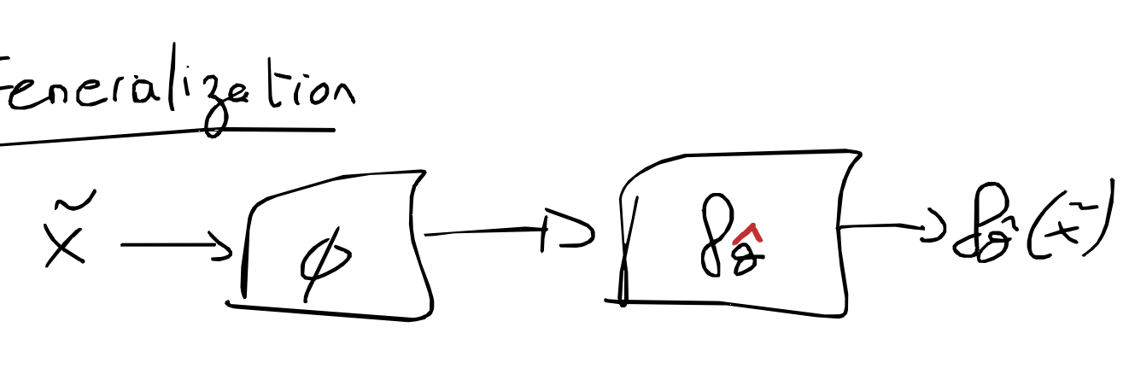
Regression: $\ell(y,\hat{y}) = \frac{1}{2}(y-\hat{y})^2$

Classification: cross-entropy

$$\ell(y,\hat{y}) = -\sum_{k=0}^{K-1} \mathbb{1}_{y=k} \log p_\theta(\hat{y})$$

③ Find $\hat{\theta}$, the "best" possible parameter by $\hat{\theta} = \arg\min_\theta R(D,X)$

Generalization



↳ What can go wrong in generalization?

I- Principles of Supervised Learning

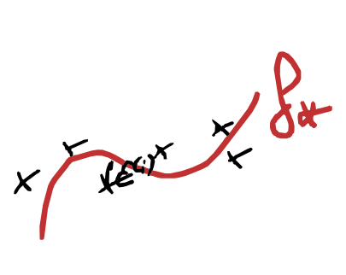
1.1. Test error in regression

Regression:

target $y^{(i)} = f_*(x^{(i)}) + \varepsilon^{(i)}$

f_*: true model (deterministic, unknown)

$\varepsilon^{(i)}$: measurement error $\begin{cases} \mathbb{E}[\varepsilon] = 0 \\ \mathbb{E}[\varepsilon^2] = \sigma^2 \end{cases}$



In ML, we build a model f_θ that is trying to approximate f_*:

$$\ell(y,\hat{y}) = \frac{1}{2}(y-\hat{y})^2$$

$$\hat{y} \triangleq \underbrace{f_\theta(x^{(i)})}_{\text{estimate of } f_*}$$ Prediction

Now, compute the expected error that the model makes on a new data $\tilde{x}$.

$$\underset{\substack{\uparrow\\ \text{dataset}}}{\mathbb{E}_{D,\varepsilon}}\Big[\big(\underset{\substack{\uparrow\\ \text{target}}}{\tilde{y}} - f_\theta(\underset{\substack{\uparrow\\ \text{test data}}}{\tilde{x}})\big)^2\Big]$$ measurement error

$\triangleq$ In average, the error of the model on a new data (not seen during training)

$\mathbb{E} \triangleq \mathbb{E}_{D,\varepsilon}$ $f_\theta \triangleq f_\theta(\tilde{x})$

$= \mathbb{E}\left[\tilde{y}^2 - 2 f_\theta \tilde{y} + f_\theta^2\right]$

$= \underset{①}{\mathbb{E}(\tilde{y}^2)} - 2\underset{②}{\mathbb{E}(f_\theta \tilde{y})} + \underset{③}{\mathbb{E}(f_\theta^2)}$ (1)

③ $\mathbb{E}[f_\theta^2] = \mathrm{var}(f_\theta) + \mathbb{E}(f_\theta)^2$

① $\mathbb{E}(\tilde{y}^2) = \mathbb{E}((f_* + \varepsilon)^2)$

$= \mathbb{E}(f_*^2) + 2\,\mathbb{E}(f_*\varepsilon) + \mathbb{E}(\varepsilon^2)$

$= f_*^2 + \underbrace{2\,\mathbb{E}(f_*)\underbrace{\mathbb{E}(\varepsilon)}_{0}}_{} + \sigma^2$

② $\mathbb{E}[f_\theta \tilde{y}] = \mathbb{E}[f_\theta (f_* + \varepsilon)]$

$= \mathbb{E}(f_* f_\theta) + \underbrace{\mathbb{E}(\varepsilon f_\theta)}_{0}$

$= f_* \mathbb{E}(f_\theta)$

$$\boxed{\mathbb{E}\left[(\tilde{y} - f_\theta)^2\right] = \underset{①}{(f_* - \mathbb{E}(f_\theta))^2} + \underset{②}{\mathrm{var}(f_\theta)} + \underset{③}{\sigma^2}}$$

Remark. Here all three terms are ≥ 0

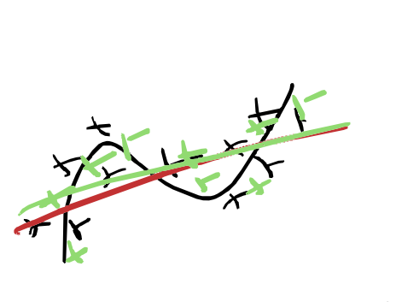
③ Irreducible error: σ^2, intrinsic error that you have in your data.

① Bias: at which point the learned model f_θ is far from truth f_*.

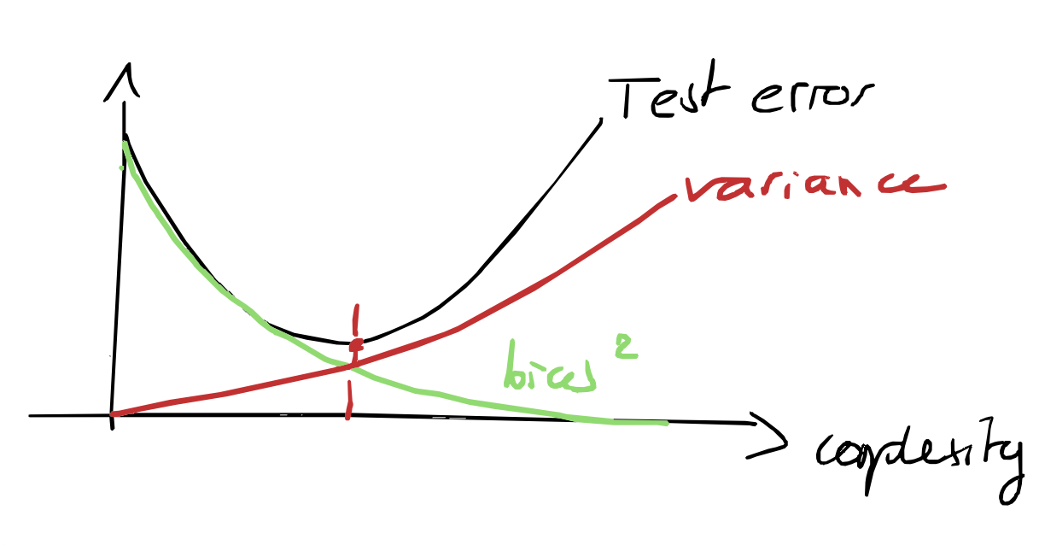
② Variance: ⊥ from truth. How sensitive your model is to the training data.

1.2 ML consequences

- "simple models" (ex linear from session 3) have high bias but low variance



- "Complex models" (ex: large neural networks with lots of parameter) have low bias but large variance.



Lesson:

Models need to be just complex enough to represent the truth but not too much to have a large variance

Here, we defined two important ML notions:

- Underfitting: large bias / low variance
- overfitting: Large Variance / Low bias

$\mathbb{E}_{D,\varepsilon}[\ell(y,\hat{y})]$ is not analytically tractable: you don't know the data distribution

So we compute instead empirical average over new data points: Test set $\tilde{x}$.

Two errors: $\begin{cases} R_{train} = R(X,\hat{\theta}) \\ R_{test} = R(\tilde{x},\hat{\theta}) \end{cases}$

Idea: Identify under/over fitting by comparing R_{train} and R_{test}.

low R_{train} but high R_{test}: overfitting model

1.3 A general cure to overfitting: Regularization

Linear regression:

$\hat{\theta} = (X^TX)^{-1}X^Ty$

If $n \simeq d$ the X^TX is not invertible and the test error blows up

↳ overfitting

We understood the problem was having too complex models. The idea of regularization is to reduce the space of functions you are looking after during optimization

$$\hat{\theta} = \arg\min_{\theta \in \Theta} \left[R(D,\theta) + \lambda P(\theta) \right]$$

regularization

penalizing "too complex" models

Remarks. ① $\lambda \geq 0$

② $\lambda = 0$ gives you back the "unregularized" problem

③ $\lambda \to +\infty$ becomes ⊥ from the training data

- $P(\theta)$ can take ≠ forms depending on the constraints you want to impose. The space Θ is being reduced by the penalty
- λ is the regularization parameter controlling the strength of the penalty. (λ is an example of hyperparameter).
- Common $P(\theta)$ are just L_p norms

$P(\theta) = \|\theta\|_p$

Ex: $P(\theta) = \|\theta\|_2$: enforces some smoothness constraints

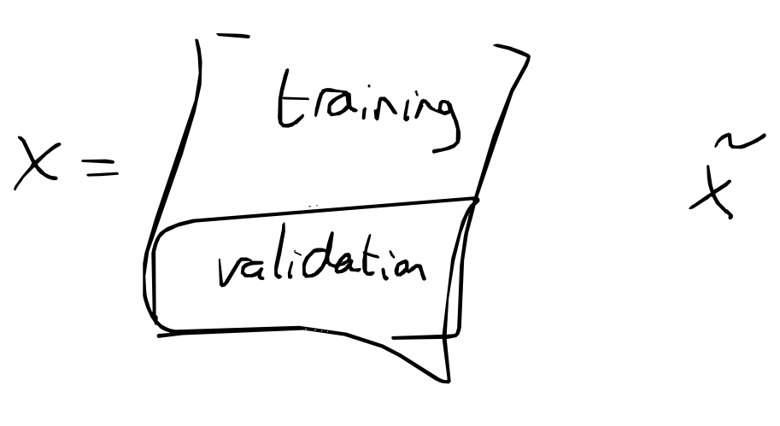
$P(\theta) = \|\theta\|_1$: enforces Sparsity constraint (bring some parameters to be 0)

With $\|\theta\|_2^2$ penalty, the linear regression becomes the Ridge regression with

$$\boxed{\hat{\theta} = \frac{1}{n}\Big(\underbrace{\frac{X^TX}{n} + \lambda I}_{}\Big)^{-1} X^Ty}$$

always invertible if $\lambda > 0$

1.4. Model selection for hyperparameters



≠ datasets in ML:

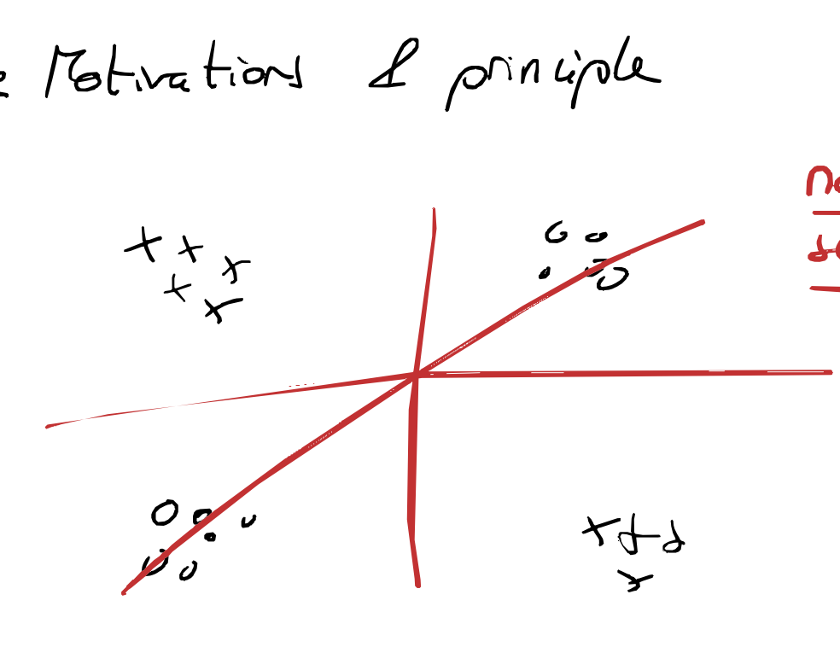
① Training set: with which you fit the parameters θ

② Validation set: A spared part of your training data that you keep to fit the hyperparameters

③ Test set: Pristine, you never fit anything on it.

II - Trees and ensembling methods

2.1. Motivations & principle



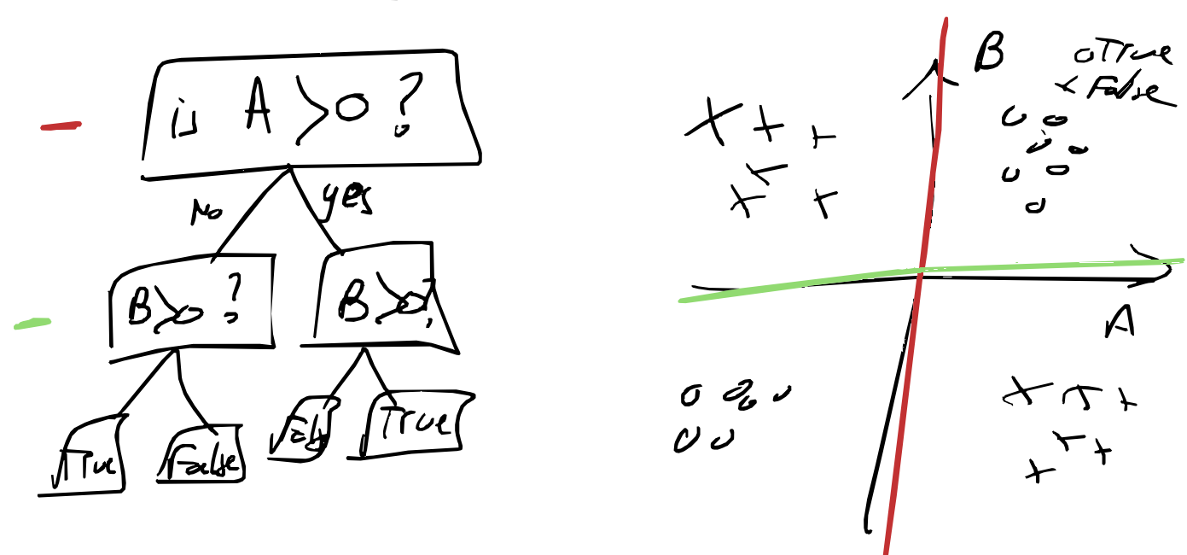
no linear separation

Session 3 linear classification is failing

Simplest non linear model is the decision tree

What is a decision tree?

① Incrementally ask binary questions about a feature (yes or no answer)



Vocabulary:

Node: question about a feature

Branch: splitting into two regions

Leaf: gives the classification or regression result.

2.2. Formal intro to DT

A given node in a parent region R_i is asking a question about feature j ⇒ split R_i into two regions:

threshold on feature j in region R_i

$R_{i1} = \{x \mid x_j < s_i^j, x \in R_i\}$

$R_{i2} = \{x \mid x_j \geq s_i^j, x \in R_i\}$

In the end, a DT is just a parametrized $f_\theta(x^{(i)})$ function with parameters $\{s_i^j\}$

To find the optimal sequence of thresholds, you minimize a loss function in each region

One choice for classification is cross-entropy and for regression is mean-squared error.

⊕ See slides

⊖ Can overfit if trees are deep, can underfit if trees are shallow

2.3. Solving overfitting with ensembling

High variance of deep decision trees.

X_i independant Random variables

$\begin{cases} \mathbb{E}(X_i) = 0 \\ \mathbb{E}(X_i^2) = \sigma^2 \end{cases}$

What is $\mathrm{var}\left(\frac{1}{n}\sum_i X_i\right)$?

$\mathrm{var}\left(\frac{1}{n}\sum_i X_i\right) = \frac{1}{n^2}\sum_i \mathrm{var}(X_i)$

$= \frac{\sigma^2 n}{n^2} = \boxed{\frac{\sigma^2}{n}}$

Think of X_i as models?

To reduce the variance, we need two ingredients:

① Make the models as ⊥ as possible

② Train a lot of them (n large)

③ Average their output.

This is the reasoning behind "bagging" in ML.